Pemrosesan Bahasa Alami

PyCaret AutoML for NLP

Martin Manullang

Institut Teknologi Sumatera
mctm.web.id

2026

OUTLINE

Peta materi pertemuan ini

1. **Stack Teknologi & Arsitektur**
2. **Pengenalan PyCaret**
3. **Dataset**
4. **Teori Preprocessing Teks**
5. **Alur Pipeline**
6. **Komparasi Model & Evaluasi**
7. **Diskusi**

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Memahami konsep dasar *pipeline* NLP pada Machine Learning tradisional.
2. Mampu melakukan tahapan ekstraksi fitur dari teks (*TF-IDF* / *Bag of Words*).
3. Memahami penerapan AutoML menggunakan `pycaret` untuk teks.
4. Mampu mengevaluasi model *traditional ML* pada kasus klasifikasi teks.

Fokus Kelas Ini

Kita akan berfokus pada pendekatan *hands-on* dan logika arsitektur modular, **menghindari boilerplate code** yang berlebihan.

Disclaimer

Peringatan Konten SARA & Eksplisit

Studi kasus pada pembelajaran kali ini menggunakan data obrolan interaksi nyata pengguna gim daring (*online game*). Oleh karena itu, Dataset yang digunakan mungkin akan memuat kata-kata **SARA, slang, kata-kata rasis**, dan bahasa yang **tidak pantas**.

Materi ini murni digunakan untuk tujuan edukasi dan penelitian pemrosesan bahasa alami.

Stack Teknologi & Arsitektur

Stack Teknologi

Library yang akan digunakan

Pendekatan: **Traditional Machine Learning via AutoML**

- `pycaret`: Untuk *training*, komparasi, dan evaluasi model secara instan.
- `pandas`: Untuk manipulasi dan eksplorasi data.
- `re` / `nltk` / `Sastrawi`: Untuk *custom text preprocessing* khusus bahasa Indonesia.

Arsitektur Kode Berbasis Modul

Pemisahan Core Logic dan Viewer

Kita **tidak akan** menumpuk eksekusi kode di dalam satu Jupyter Notebook.

File `.py` (Core Logic)

Seluruh logika ekstraksi, inisialisasi `pycaret`, komparasi, dan ekspor model.

- `config.py`
- `preprocess.py`
- `train.py`

File `.ipynb` (Runner & Viewer)

Hanya difungsikan sebagai *runner* presentasi. Memanggil modul `.py` dalam satu baris, dan menampilkan tabel atau plot grafik hasil.

Pengenalan PyCaret

1. Apa itu PyCaret?

PyCaret adalah *framework open-source* berbasis Python dengan konsep *low-code* untuk memfasilitasi *Automated Machine Learning* (AutoML).

- **Pengembang Utama**: Diciptakan dan dipelihara oleh **Moez Ali** sejak tahun 2019.
- **Tujuan Pokok**: Mengotomatiskan eksperimen *machine learning end-to-end*. Mulai dari preparasi data otomatis, *training*, komparasi, *hyperparameter tuning*, analisis *metrics*, hingga ekspor / *deployment*.

2. Mengapa PyCaret Sangat Membantu?

- **Low-Code Efficiency**: Secara drastis dapat menggantikan ratusan hingga ribuan baris kode *boilerplate* hanya dengan hitungan 1 atau 2 baris *syntax*.
- **Produktivitas Eksperimen**: Eksperimen puluhan model komparasi algoritma kompleks dari beragam basis librari (`sklearn`, `xgboost`, `catboost`) bisa berjalan instan.
- **Dinding Keamanan Standar**: Praktik pipelining bawaannya mampu mencegah bahaya laten ML sepert *data leakage* dan anomali konversi selama rotasi *K-Fold Cross-Validation*.
- **Berorientasi Solusi (*Beginner Friendly*)**: Pengguna / *Data Scientist* dapat terfokus meneliti kehandalan interpretasi ketimbang pusing mengecor konstruksi algoritma.

3. Fitur Utama & Cakupan Algoritma

Sistem Fundamental Eksperimen PyCaret:

1. `setup()`: Katalisator awal lingkungan. Mentransformasi seluruh vektorisasi teks menjadi data pelatihan siap pakai.
2. `compare_models()`: Arena adu jotos akurasi antarmodel.

Ruang Lingkup Estimator (Model Algoritma):

- PyCaret memborong banyak algoritma tradisional *state-of-the-art* untuk bertarung secara bersamaan, beberapa *heavy-hitters*-nya antara lain: ***Light Gradient Boosting Machine*** (`lightgbm`), ***Random Forest*** (`rf`), ***Decision Tree*** (`dt`), ***Support Vector Machines*** (`svm`), ***K-Nearest Neighbors*** (`knn`).

Dataset

Detail Dataset NLP

Indonesian Chat Dataset (Roblox & Minecraft)

Sumber: Kaggle (*Indonesian Chat Dataset*)
Karakteristik Teks: Obrolan *gamer* yang **kotor**. Penuh dengan *leetspeak*, singkatan ekstrem, dan *slang* lokal.

Informasi Dataset:

- **Ukuran dataset**: 10.702 baris $\times$ 3 kolom
- **Kolom**: `['id', 'chat', 'label']`

Tugas: Multiclass Classification (Distribusi Label)

- `violence` : 2.983 (27.9%)
- `neutral` : 2.729 (25.5%)
- `racist` : 2.506 (23.4%)
- `harassment` : 2.484 (23.2%)
- **TOTAL** : **10.702 baris**

Contoh Data

Berikut adalah cuplikan data teratas:

id	chat	label
0	main mu kek tai cok	violence
1	user telat ngasih tau elu edan sarap gue bergaul elu	violence
2	kadang berfikir percaya tuhan jatuh berkalikali kadang tuhan ninggalkan oran...	neutral
3	user user aku\n\nku tau matamu sipit diliat	racist
4	capek deh ketemu kaum cina kapir gini match	racist

Teori Preprocessing Teks

1. Apa itu Text Preprocessing?

Mengapa teks harus dibersihkan?

Text Preprocessing adalah tahapan krusial dalam NLP untuk mengubah teks mentah menjadi format yang dapat dipahami oleh model *Machine Learning*.

Alasan Utama:

- Teks dari internet (terutama *chat*) penuh dengan "noise".
- Komputer tidak memahami teks alfabetikal, melainkan **angka**.
- Teks yang bersih akan **mengurangi dimensi fitur** (jumlah kata unik) secara sangat drastis dan **meningkatkan akurasi** model.
- Prinsip *"Garbage In, Garbage Out"* - Sebaik apapun algoritma komparasi, jika kualitas teks buruk maka hasil prediksi gagal.

2. Tahapan Klasik Preprocessing Teks

Secara teori, alur pembersihan teks tradisional seringkali melewati 6 langkah berikut secara berurutan:

1. **Text Cleaning** (Pembersihan Teks Dasar)
2. **Normalization** / **Slang Words Handling** (Normalisasi)
3. **Tokenization** (Tokenisasi)
4. **Stopword Removal** (Penyaringan Stopword)
5. **Stemming** / **Lemmatization** (Pencarian Kata Dasar)
6. **Text Representation** / **Vectorization** (Vektorisasi TF-IDF)

3. Text Cleaning (Pembersihan Awal)

Lowercasing & Punctuation Removal

Lowercasing

- Mengubah semua huruf menjadi huruf kecil.
- Menyeragamkan kosakata: "Toxic", "TOXIC", dan "toxic" akan digabungkan menjadi satu identitas variabel kata.
- *Contoh*: "DASAR NOOB!" $\rightarrow$ "dasar noob!"

Punctuation & Special Character Removal

- Menghapus tanda baca (!, ?, ., ,) dan karakter emoji / simbol ASCII.
- Karakter non-alfabet terkadang jarang memiliki makna prediktif kuat pada ML Tradisional (*Term Frequency*).
- *Contoh*: "halo!!! apa kabar??" $\rightarrow$ "halo apa kabar"

4. Text Cleaning (Lanjutan)

Menangani URL, Mention, dan Hashtag

Data media sosial/gim sering memuat elemen teknis sistem yang tidak memiliki makna leksikal *human-language*:

- **Menghapus URL/Link**: Tautan situs seperti `http://...` atau `www.` dihapus karena merupakan *noise* mutlak.
- **Menghapus Mention**: Kata yang diawali `@`. Biasanya dinormalisasi menjadi token netral **[USER]**. *Di dataset kita, hal ini sudah disensor menjadi kata "user"*.
- **Menangani Hashtag**: Tanda `#` dihapus namun teksnya dipertahankan, karena misal kata `#boikot` bermakna penting dalam teks opini.

5. Normalization & Slang Handling

Tantangan Teks Gamer & Media Sosial

Data chat tidak menggunakan bahasa baku. Tahap ini sangat krusial:

- **Leetspeak Transformation**: Memetakan kata beralfanumerik ke abjad absolut.
  - *Contoh*: "m4k4n" $\rightarrow$ "makan", "b99" $\rightarrow$ "bangg"
- **Repeated Characters Removal**: Memotong karakter yang memanjang tidak wajar (*elongated words*).
  - *Contoh*: "noooobbbb bangggeetttt" $\rightarrow$ "noob banget"
- **Slang Dictionary Mapping**: Kamus pemetaan manual (*alay map*) untuk memperbaiki ejaan.
  - *Contoh*: "yg", "sgt", "kuy" $\rightarrow$ "yang", "sangat", "ayo"

6. Tokenization

Pemecahan Teks Menjadi Unit Kecil

Tokenization adalah proses memecah kalimat utuh menjadi unit-unit individu teks yang lebih diskrit yang disebut dengan *token* (kata).

Contoh Tokenisasi Kata (Word Tokenization)

Kalimat: "Bocah ini sangat toxic sekali"
→ `["bocah", "ini", "sangat", "toxic", "sekali"]`

Fungsi dari tahap ini adalah agar model (seperti Scikit-Learn atau PyCaret nantinya) dapat menghitung statistik frekuensi tiap kata satu per satu pada tahap representasi vektor.

7. Stopword Removal

Menyaring Kata Kebisingan Ekstrim

Stopwords adalah kata-kata sambung atau umum yang frekuensi kemunculannya ekstrim, **namun** secara semantik tidak membedakan polaritas kelas pada kalimat.

- **Contoh (Indonesia)**: "yang", "dan", "di", "ke", "dari", "ini", "itu".
- Menghapus stopwords akan memangkas dimensi memori pelatihan algoritma secara dramatis tanpa mengorbankan konteks klasifikasi.

Contoh Filtering Stopword

Awal: `["saya", "ingin", "makan", "di", "restoran", "itu"]`
Akhir: `["makan", "restoran"]`

8. Stemming vs Lemmatization

Dua Pendekatan Mengurai Imbuhan

Tujuannya yaitu memadatkan atau menyatukan variasi dan variabilitas imbuhan menjadi 1 representasi kata dasar.

Stemming

- Memotong otomatis awalan/akhiran menggunakan *heuristic rules* (kasar).
- Sangat ringkas, cepat, namun hasil katanya bisa tidak memiliki arti.
- *memperbaiki* $\rightarrow$ *baik*

Lemmatization

- Resolusi leksikal analisis kamus dan morfologi kata asal (lemma).
- Sangat komputasional / lambat, ekstensif gramatikal, tapi *100% valid*.
- *better* $\rightarrow$ *good*

9. Dilema Stemming Bahasa Indonesia

Mekanik *Library* Sastrawi

Bahasa Indonesia memiliki morfologi rumit. *Lemmatization* sangat berat diterapkan karena minimnya ketersediaan NLP teroptimasi.

- Peneliti Python lokal menggunakan *library* **Sastrawi** / **PySastrawi** untuk urusan **Stemming** Indonesia.
- Implementasinya mengacu pada turunan algoritma *Nazief & Adriani*.
- **Permasalahannya** pada kasus dataset gamer / chat *slang*: Stemmer formal gagal melakukan stemming pada kalimat yang bukan bahasa baku / KBBI. Oleh sebab itu penanganan slang wajib di nomor satu-kan.

10. Text Feature Representation

Bag of Words & TF-IDF

Seketika preprocessing selesai, hasil akhir corpus masuk ke babak matematis mengubah tokenisasi vektor huruf $\rightarrow$ menjadi angka riil desimal.

- **Bag of Words (BoW)**: Mendata total statistik frekuensi setiap kosakata dokumen. *Term yang paling sering tampil memegang kendali terbanyak, mengabaikan konteks di kalimat lain.*
- **TF-IDF** (*Term Frequency - Inverse Document Frequency*): Algoritma standar industri untuk menskor (rating) pada kata.
    - Misal, kosakata obrolan X sering muncul kuat di baris "toxic", **TAPI JARANG** diutarakan pemain di baris netral, nilai kata itu **dikali berlipat ganda karena ia merupakan kata unik per kelas.**

Alur Pipeline

Tahapan Pipeline (1-2)

Dari Pembersihan ke Setup

1. **Custom Text Cleaning**
   - Teks *gamer* butuh pembersihan khusus.
   - Menangani *leetspeak* angka dan *slang* agar kata-kata tidak terbuang atau kehilangan maknanya.

2. **AutoML Setup**
   - Menggunakan `pycaret.classification.setup`.
   - Konfigurasi setup untuk melakukan transformasi *TF-IDF* (*Term Frequency - Inverse Document Frequency*) secara otomatis.

1. Bedah Fungsi `setup()`

Gerbang Utama Eksperimen

Fungsi `setup()` adalah langkah terpenting dalam PyCaret yang wajib diinisialisasi pertama kali.

- **Fungsi Pokok**: Mengambil variabel dataframe *pandas* dan merekayasa skema lingkungan ML secara absolut.
- Mengelola strategi *train-test split* secara adil dengan menyamakan rasio distribusi kelas minoritas dan mayoritas (*Stratified K-Fold*).
- Mentransformasi fitur teks menjadi numerik berdimensi tinggi sebelum dikonsumsi oleh algoritma menggunakan *TF-IDF text embedding*.

2. Hyperparameter pada `setup()` (1/2)

Konfigurasi Inti & Teks

Dalam PyCaret, fungsi `setup()` memiliki belasan argumen. Berikut beberapa konfigurasi penting:

Parameter Wajib

- `data=df_chat` → Mendeklarasikan sumber *dataset* dari objek *pandas* di memori.
- `target='label'` → Menentukan nama spesifik kolom yang ingin diprediksi klasifikasinya.

Parameter Reproducibility

- `session_id=42` → Mengunci *random seed state* agar pembagian acak konsisten (*reproducible*) setiap kali *code* dijalankan di mesin siapapun.

Parameter Fitur Teks Khusus NLP

- `text_features=['chat']` → Memberitahu mesin tentang eksistensi kolom berisi teks paragraf mentah yang wajib dibedah.
- `text_features_embedding_method='tf-idf'` → Menugaskan ekstraktor algoritma TF-IDF (*Term Frequency Inverse Document Frequency*) ke kolom teks di atas. (*Bisa diubah ke 'bow' untuk sekadar hitung frekuensi kemunculan*).

3. Hyperparameter Lanjutan `setup()` (2/2)

Strategi Validasi, Hardware, & Preprocessing Ekstra

Strategi Training & Hardware

- `train_size=0.8` → Porsi pemisahan data latih (80%) vs data uji rahasia (20%).
- `fold=5` & `fold_strategy='stratified'` → Memecah validasi silang (CV) rata pada semua lipatan kelas.
- `use_gpu=True` → Mengalihkan *workload* matematika tensor ekstrim ke RAM Kartu Grafis (NVIDIA/CUDA).
- `n_jobs=-1` → Mengerahkan paksa efisiensi 100% utilitas *core thread* CPU spesifikasi server.

Parameter *Secret Weapon* Lainnya

PyCaret sanggup menyulap manipulasi data tingkat dewa hanya dengan mengatur bendera (*flags*):

- `normalize=True` → Meratakan rasio skala simpangan baku fitur numerik ekstrem *(Z-Score / MinMax)*.
- `pca=True` → Mengompresi dimensi kolom ribuan teks (TF-IDF) jadi puluhan vektor utama (*Dimensionality Reduction*).
- `fix_imbalance=True` → Otomatis menduplikasi (*Oversampling* SMOTE) target kelas dataset yang jumlah minoritasnya menyedihkan!
- `ignore_features=['id']` → Membutakan model dari kolom irrelevan agar tidak ikut difiturisasi.

3. Membaca Output Tabel `setup()` (Bagian 1)

Saat `setup()` dieksekusi, panel *dashboard* teratas akan memberi laporan parameter komputasi. Parameter terpenting antara lain:

- **Target type** (Multiclass): Sistem otomatis mendeteksi total kategori label target melebihi dari 2 variasi kelas (*binary*).
- **Target mapping**: `{harassment: 0, neutral: 1, racist: 2, violence: 3}`. Otomatis (*One-Hot Encoder*) menyandi label teks menjadi nominal (0-3).
- **Original data shape**: Total ukuran keranjang awal **(10.702 baris, 2 kolom)**.
- **Transformed data shape**: Menjadi **(10.702 baris, 18.117 kolom fitur)**! Ukuran ruang ini meledak murni dari *vocabulary token* karena pemrosesan TF-IDF kata. Tiap sel menampung frekuensi *rating*!

4. Membaca Output Tabel `setup()` (Bagian 2)

- **Transformed train set shape**: Menyerap **8.561** obrolan ke ring (80% data primer *train* untuk menggembleng "kerangka otak" model).
- **Transformed test set shape**: Mengunci rahasia sisa proporsi **2.141** obrolan (20% hold-out). Pintu *test set* ini haram hukumnya dilihat algoritma ketika masa *training*, berfungsi vital sebagai soal ujian murni objektif.
- **Fold Generator** (StratifiedKFold): Memastikan lipatan per kelas imbang.
- **Use GPU** (True): Mengalirkan iterasi pembagian tensor NLP dan algoritma tangguh lewat unit prosesor sentral grafis VRAM (misal CUDA) yang jauh melampaui paralelisasi CPU biasa.

5. Konsep 5-Fold Cross Validation

Mengapa tidak cukup 1 kali training?

Dalam parameter awal, kita mengatur sistem siklus ditiup menjadi 5 balikan rotasi (`Fold=5`).

- **Prinsip**: Data primer *train set* **dibagi** / **di-iris adil jadi 5 patahan**.
- Model melatih pengetahuannya pada **4 patahan awal**, lalu dihantam *ujian rahasia* mendadak di **1 patahan sisa** ujung.
- **Proses ini digulirkan & diulang** dengan pergantian posisi peta *training* yang terus bergeser.
- **Tujuan Pokok**: *Machine Learning* tidak boleh hafal lokasi data pola semu. Kemampuannya harus tervalidasi jendral ke dimensi apa pun dalam distribusinya (membuang *over-lucky metric* atau bias kebetulan tinggi).

6. Diagram Visual K-Fold Cross Validation

Iterasi 1	Test Fold 1	Train Fold 2	Train Fold 3	Train Fold 4	Train Fold 5
Iterasi 2	Train Fold 1	Test Fold 2	Train Fold 3	Train Fold 4	Train Fold 5
Iterasi 3	Train Fold 1	Train Fold 2	Test Fold 3	Train Fold 4	Train Fold 5
Iterasi 4	Train Fold 1	Train Fold 2	Train Fold 3	Test Fold 4	Train Fold 5
Iterasi 5	Train Fold 1	Train Fold 2	Train Fold 3	Train Fold 4	Test Fold 5

Data Latih (80% Train Fold) | Uji Tersembunyi (20% Validasi)

Skor final parameter (*Akurasi, F1*, dll.) keluar sebagai nilai rata-rata (*Mean Out-Of-Fold*) mutlak, dan **bukan dari iterasi tebakan "terhoki"/terbaiknya**!

Tahapan Pipeline (3-5)

Komparasi Model hingga Deployment

3. **Model Arena**
   - Membandingkan (*compare*) algoritma tradisional (misal: SVM, CatBoost, LightGBM).
   - Mencari algoritma dengan Akurasi dan F1-Score teringgi.
4. **Evaluation**
   - Menganalisis hasil dari model terbaik.
   - Melihat pola dari *Confusion Matrix* dan *Feature Importance* (kata apa yang paling memicu suatu label, misal kata untuk *toxic*).
5. **Finalize**
   - Membungkus seluruh *pipeline* utuh (preprocessing, vektorisasi TF-IDF, model) menjadi file statis (`.pkl`).

Komparasi
Model & Evaluasi

1. Tinjauan Algoritma Klasik (DT & RF)

Decision Tree & Random Forest

Fungsi `compare_models()` akan mengadu belasan algoritma. Berikut profil algoritma andalan:

`dt` (Decision Tree)

Mekanisme: Membuat aturan *if-then* bercabang seperti algoritma kuesioner *Yes/No* atau Diagram Alir (*Flowchart*).

Kelebihan/Kekuranga: Pengambilan keputusannya sangat transparan dan **mudah dimengerti manusia**, namun amat rentan terhadap *overfitting* (terlalu menghafal data letak *training* ketimbang memahami pola asli).

Analogi: Seperti bos perfeksionis yang menghafal mati prosedur kaku, tapi gagap saat ada kasus baru.

`rf` (Random Forest)

Mekanisme: Evolusi (*ensemble*) dari DT. Alih-alih 1 pohon rapuh, RF menumbuhkan "hutan" berisi ratusan pohon keputusan berukuran mini secara acak, dan setiap klasifikasi akhir didapat via ***voting*** (suara terbanyak rata-rata).

Kelebihan: RF jauh lebih **stabil dan konsisten**.

Analogi: Daripada tanya 1 dokter spesialis yang bisa salah (DT), lebih baik kumpulkan 100 dokter umum lalu ambil diagnosis suara terbanyak (RF)!

2. Algoritma Jarak & Pemisah (KNN & SVM)

`knn` (K-Nearest Neighbors)

Mekanisme: Menebak sentimen teks baru berdasarkan **kedekatan jarak grafis** K-jumlah teks "tetangga terdekat"-nya.

Kekurangan Vital: Pemalas di awal, tapi tersiksa di akhir. Ia *lazy learning* menahan *semua* observasi matematis di memori sehingga performanya **menurun drastis** dihantam dataset teks lebat.

Analogi: Anak gaul yang sifat dan hobinya selalu dipengaruhi rata-rata 5 teman tongkrongannya.

`svm` (Support Vector Machines)

Mekanisme: Menarik garis tembok (*hyperplane*) mutlak sebagai **jurang pembatas** antar satu label spasial dengan label lainnya secara matematis ekstrim.

Kelebihan Utama: Sangat **kokoh, akurat, dan merajai klasifikasi dunia NLP** berskalabilitas tinggi karena tahan banting memproses miliaran dimensi ekstraksi teks TF-IDF.

Analogi: Membangun rentangan tembok Berlin raksasa untuk memisahkan kota perusuh vs kota damai.

3. State-of-the-Art: Gradient Boosting

`lightgbm` (Light Gradient Boosting Machine)

Sejarah: Mahakarya *open-source* inisiatif **Microsoft** yang sering kali merajai puncak kompetisi AI *Kaggle*. Algoritma level Dewa untuk dataset gajah.

Kehebatan: Berfokus pada efisiensi optimasi RAM yang sangat pelit (*leaf-wise tree growth*), menjadikannya sangat agresif, brutal, cepat, dengan akurasi terpresisi. Sangat *recommended* untuk AutoML.

Rahasia Mekanik (*Boosting*)

Berbeda dengan `Random Forest` (RF) yang membina ratusan pohon secara serentak & independen, metode *Boosting* itu mendidik model secara **Berantai**.

Ia membangun 1 pohon awal. Lalu disusul pohon kedua yang secara dramatis dan eksklusif **hanya difokuskan untuk memperbaiki error kelemahan dari pohon pertama!** Begitu terus menerus.

Analogi: Tim estafet lari di mana pelari kedua mati-matian mengebut guna menutupi keterbelakangan pelari pertama!

4. Membaca Scoring Metrics (Akurasi & Akal-akalan)

PyCaret mengurutkan model berdasarkan *Metrics*. Mari kita bedah hati-hati!

Accuracy (Akurasi)

Mengukur persentase rasio (%) tebakan model yang benar mutlak.

Awas, Akurasi itu Buta! Jika data cacat (99% Netral, 1% Toxic), model bodoh yang menebak SEMUA teks sebagai Netral akan mendapat akurasi ajaib 99%, memalsukan kesuksesan evaluasi!

AUC (*Area Under the ROC Curve*)

Menghitung efektivitas model membedakan sebaran dua sisi polaritas kelas agar jangan tumpang tindih.

- Skor **1.0** → Dewa *machine-learning*.
- Skor **0.5** → Setara performa melempar koin (kebetulan acak murni).

5. Precision, Recall, & F1 Score

Triumvirate Evaluasi Data Imbalanced

Precision

Dari total yang dituding "Rasis", berapa yang **benar-benar asli** rasis?

(Menuntut kehati-hatian sistem menghindari salah tangkap).

Recall

Dari jangkauan 100 pesan rasis *original*, terjaring **seberapa luas** oleh model?

(Menyapu bersih, biarpun *false alarm* ikut diangkut).

F1 Score

Nilai rata-rata penengah (*gold standard*) di antara Recall dan Precision. **Jadikan skor F1 prioritas utama di kehidupan nyata!**

6. Contoh Perhitungan: Kasus Kebencian (1/2)

Studi Kasus: Kita memiliki **100 chat** ujaran kebencian asli di dalam data rahasia (*Test Set*).

Skenario A (Super Hati-hati = High Precision)

Model A menembak 10 pesan sebagai kebencian.

- Dari 10 tebakan, **10-nya 100% BENAR asli**.
- **Precision** = 100% (Sempurna tidak salah tangkap)
- **Recall** = Namun dari 100 penjahat asli, ia cuma berhasil menjaring 10. (Recall = 10% - Gagal total melacak sisa 90 penjahat lain!)

Skenario B (Super Serok = High Recall)

Model B menembak **200 pesan** sekaligus sebagai kebencian.

- Dari 200 tembakan, ia berhasil mengangkat ke-100 kotoran. **Recall = 100%** (Menyapu bersih).
- **Precision** = Total tebak 200, cuma benar 100. (Precision 50%, artinya setengah warga korbannya ditangkap sembarangan!)

7. Contoh Perhitungan: Kasus Kebencian (2/2)

Kita tidak mau model yang kerjanya asal tangkap rakyat (Skenario B), tapi enggan pula mempekerjakan detektif yang gagal melacak sisa pelapor kejahatan (Skenario A). **Di situlah F1 Score lahir**.

Rumus Harfiah F1-Score

$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$

Ilustrasi Hasil Imbang Model C:

- Model C menebak 110 tebakan kebencian.
- Ia berhasil menangkap 90 kotoran murni (Recall **90%**).
- Dari 110 tembakan, meleset 20 ke orang tak bersalah (Precision: 90 / 110 = **81.8%**).

Putusan PyCaret

Skor F1 Model C: $= 2 \times \frac{0.818 \times 0.90}{0.818 + 0.90} \approx$ **0.85** (atau 85%)

Data Scientist akan selalu memperebutkan dan memenangkan Model C di turnamen karena performa keseimbangannya yang jauh lebih realistis!

8. Indikator Tambahan (Kappa, MCC, TT)

Kappa & MCC (*Matthews Correlation Coefficient*)

Variasi *scoring threshold* penyeimbang perhitungan "kebetulan asal tebak dadu", amat kebal rentetan kelas data miring.

- Nilai **1.0** $\rightarrow$ Sempurna mutlak.
- Nilai $\leq$ **0.0** $\rightarrow$ Lebih hancur daripada kemujuran menebak acak.

TT (Sec) (*Training Time*)

Jumlah waktu dimensi sekon yang dihanguskan komputer merakit pondasi arsitektur dari 0 buat 1 siklus *Fold* CV spesifik utuh.

Analogi: Apakah naik akurasi mikroskopis F1

0.02% masih setimpal membakar umur

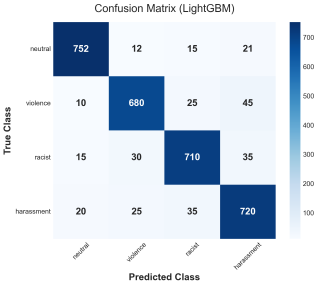
durasi server 3 jaman buat training kodenya?

7. Menyibak Matriks Kebingungan

Membaca Plot Confusion Matrix

Fungsi dasar `plot_model(best, 'confusion_matrix')` akan menyibak visualisasi tabir peraduan antara **Tebakan Model vs Jawaban Hakiki** (*True Class*).

- Kotak **diagonal (kiri atas ke kanan bawah)** adalah titik intersep tebakan yang murni sempurna (Akurat).
- **Analisis**: "*Terlihat model amat naif, gemar salah menebak kata harassment ke kotak violence! Pasti ada kosakata Toxic yang saling mengikat kedua ranah kalimat itu!*"



8. Memaksimalkan Potensi (Hyperparameter Tuning)

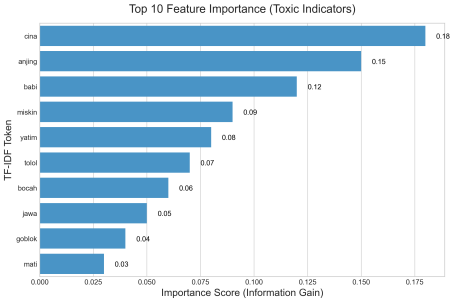
Meskipun `compare_models()` mensortir algoritma terbaik, mesin tersebut belum mengeluarkan performa mutlaknya.

- Sintaks komando `tune_model(best, optimize="F1")` menyetir utilitas automasi *Grid/Random Search* memutar kombinasi kenop tuas matematis arsitektur algoritma (contoh: kedalaman percabangan pohon RF, *learning-rate* LightGBM).
- Skema *tuning* menggulir silang kombinasi setelan agar mencapai titik harmonis rekor akurasi **F1-Score** setinggi mungkin di ujung jurang batas ketahanan model agar tidak *overfitting*.

9. Interpretability (Feature Importance)

Kita bisa mewawancarai alasan algoritma memvonis suatu ujaran kotor melalui `plot_model(tuned, 'feature')`!

- **Insight**: "Ternyata token kata: *cina, anjing, babi, yatim, tolol* konsisten menguasai puncak peringkat koefisien matematisnya!"
- Jika *noise stopword* (cth: "kamu") juara 1, saatnya siklus dibongkar *cleaning* ulang!



10. Finalize & Export Model Tunggal

Mengawetkan "Otak" Artificial Intelligence

- Fase pelantikan `finalize_model(tuned)` memaksa pakem algoritma mengawinkan paksa porsi irisan % Data Uji Rahasia (yang tabu diintip sedari awal iterasi) melebur kembali menyatu ke wadah *Train*. Ia akan melahap mutlak absolut 100% bongkahan rekaman total *dataset* demi adaptabilitas terjun ke dunia siber nyata (*The True Final Form*).
- Eksekutor kompilasi akhir `save_model(final, 'text_model_v1')` menyolder **semua perakitan kompleks multi-lapisan**: dari (1) Ekstraksi Kamus TF-IDF Teks, (2) Transformator Target One-Hot Encoder, terbungkus hingga (3) Algoritma Klasifikasi final LightGBM/RF utuh . . . ke dalam wujud file saku biner statis ekstensi `.pkl` **(Pickle)**.

11. Deployment Antarmuka ke Hugging Face (1)

Kecerdasan *Artificial Intelligence* lumpuh fungsinya jika mustahil dipakai uji obrolan (*interactive inference*) oleh warga masyarakat awam.

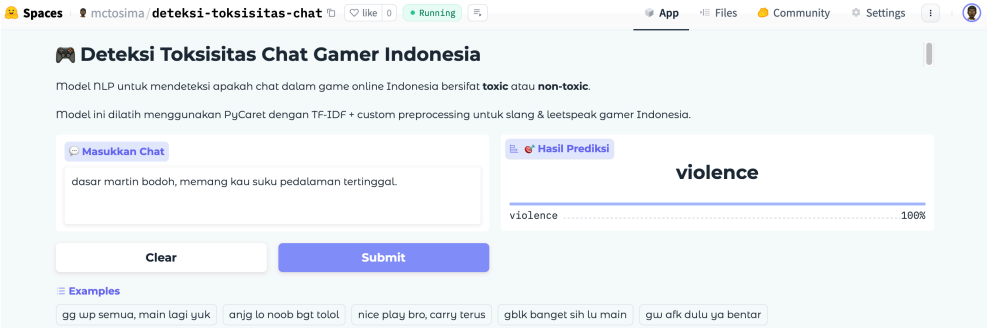
- **Hugging Face Spaces** menganugerahkan slot arsitektur komputer maya kontainer berbasis Linux gratis mengudara non-stop buat *hosting* portal *Machine Learning*.
- *Engineer* menugaskan antarmuka tatap muka elegan *low-code* menawan (*Web UI Script*) `app.py` berbasis gubahan *framework* cepat **Gradio** atau **Streamlit**.
- Web UI ini meresonansikan panggilan pemuatan (*loading state*) *Pickle* saku resep model `.pkl` keras kita menuju aliran kesibukan *Random Access Memory* (RAM) server kontainer *cloud* HF.

12. Peluncuran Global HF Spaces (2)

- Guna menjamin kontainer awan *Linux Server* fasih mengoperasikan logika kita tanpa malfungsi eror, manifes daftar belanjaan resep pustaka bernomenklatur `requirements.txt` harus dinaik-sertakan.
- Mesin otomatis meng-unduh dan menanam persis jejak paket setara *laptop programmer*-nya (Misal mencanangkan entri paku: `pycaret==3.0.4`, `gradio`, `pandas`) tepat sesaat jelang menekan tombol hijau *Boot ON*.
- **Misi Paripurna**: Sedetik usai komoditas sirkuit Git *Push* terkompilasi nyala, sang pionir rekayasa NLP **resmi memiliki portal *endpoint* URL publik aplikasi sensor pendeteksi kata-kata maki *gamer* mahakaryanya mengudara ke seluruh benua planet** siap dilibas berjuta obrolan manusia!

13. Hasil Deployment (Live UI)

https://huggingface.co/spaces/mctosima/deteksi-toksisitas-chat



Diskusi

Hands-on Time

Silakan buka teks editor dan `workshop_sesi1.ipynb` kalian. Kita akan mulai dengan membuat kerangka modulenya terlebih dahulu!